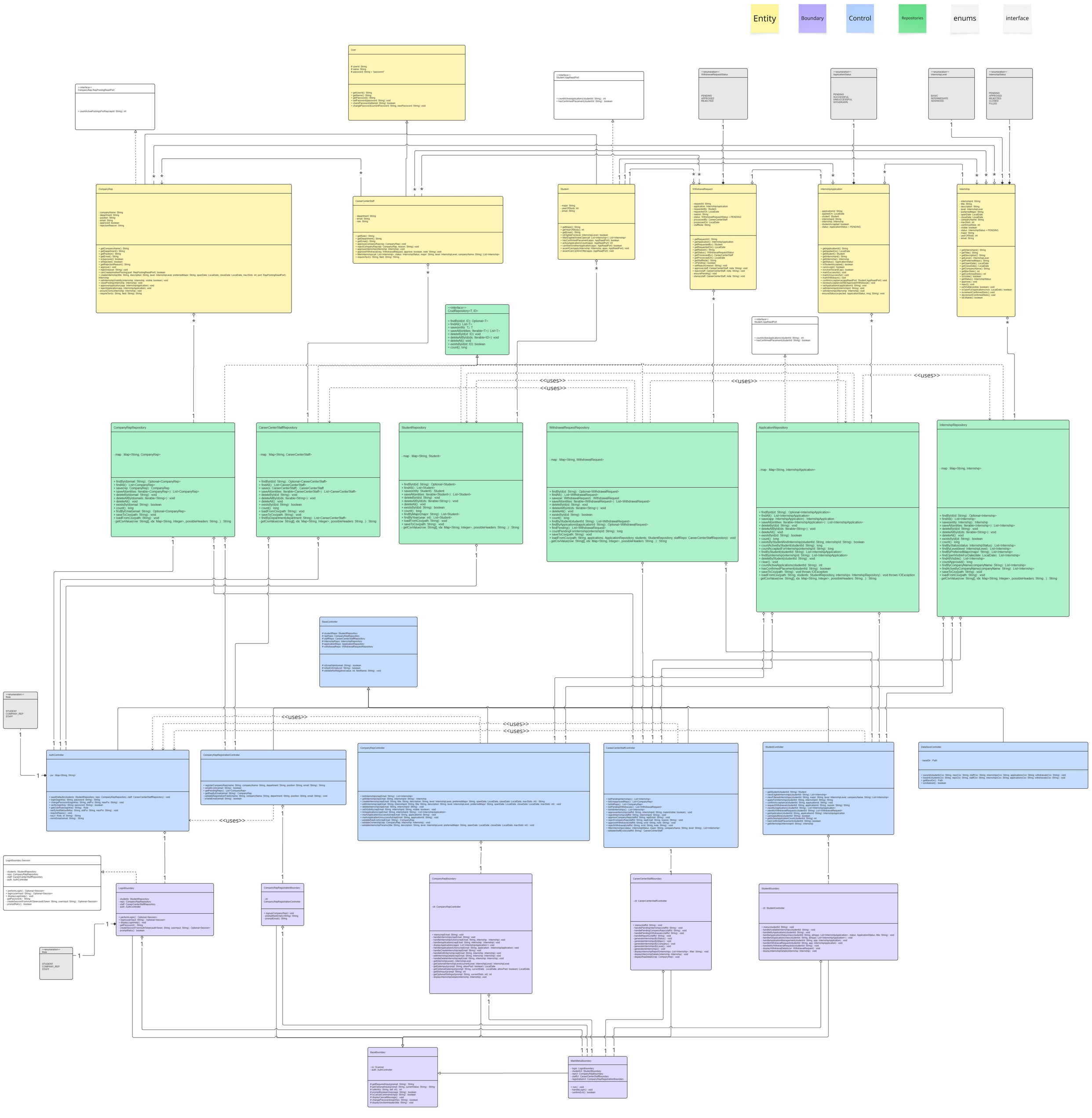
Entity
Boundary
Control
Repositories
enums
interface
User
userId: String
name: String
password: String = "password"
+ getUserId(): String
+ getName(): String
+ getPassword(): String
+ setPassword(password: String): void
+ checkPassword(attempt: String): boolean
+ changePassword(currentPassword: String, newPassword: String): void
<<interface>>
CompanyRep.RepPostingReadPort
+ countActivePostingsForRep(repId: String): int
<<interface>>
Student.AppReadPort
+ countActiveApplications(studentId: String) : int
+ hasConfirmedPlacement(studentId: String) : boolean
<<enumeration>>
WithdrawalRequestStatus
PENDING
APPROVED
REJECTED
<<enumeration>>
ApplicationStatus
PENDING
SUCCESSFUL
UNSUCCESSFUL
WITHDRAWN
<<enumeration>>
InternshipLevel
BASIC
INTERMEDIATE
ADVANCED
<<enumeration>>
InternshipStatus
PENDING
APPROVED
REJECTED
CLOSED
FILLED
CompanyRep
- companyName: String
- department: String
- position: String
- email: String
- approved: boolean
- rejectionReason: String
+ getCompanyName(): String
+ getDepartment(): String
+ getPosition(): String
+ getEmail(): String
+ isApproved(): boolean
+ isRejected(): boolean
+ getRejectionReason(): String
+ approve(): void
+ reject(reason: String): void
+ canCreateAnotherPostingLimit: RepPostingReadPort): boolean
+ createInternship(title: String, description: String, level: InternshipLevel, preferredMajor: String, openDate: LocalDate, closeDate: LocalDate, maxSlots: int, port: RepPostingReadPort): Internship
+ setInternshipVisibility(internship: Internship, visible: boolean): void
+ closePosting(internship: Internship): void
+ approveApplication(app: InternshipApplication): void
+ rejectApplication(app: InternshipApplication): void
+ ensureOwns(internship: Internship): void
- requireText(s: String, field: String): String
CareerCenterStaff
- department: String
- email: String
- role: String
+ getRole(): String
+ getDepartment(): String
+ getEmail(): String
+ approveCompanyRep(rep: CompanyRep): void
+ rejectCompanyRep(rep: CompanyRep, reason: String): void
+ approveInternship(internship: Internship): void
+ approveWithdrawal(req: WithdrawalRequest, approve: boolean, note: String): void
+ filterInternships(all: List<Internship>, status: InternshipStatus, major: String, level: InternshipLevel, companyName: String): List<Internship>
+ requireTextIn(s: String, field: String): String
Student
- major: String
- yearOfStudy: int
- email: String
+ getMajor(): String
+ getYearOfStudy(): int
+ getEmail(): String
+ isEligibleForJobs(): boolean
+ isEligibleForInternsOpen(all: List<Internship>): List<Internship>
+ hasConfirmedPlacement(app: AppReadPort): boolean
+ activeApplicationsCount(app: AppReadPort): int
+ canStartAnotherApplication(app: AppReadPort): boolean
+ assertCanApply(internship: Internship, app: AppReadPort): void
+ assertCanConfirmOffer(app: AppReadPort): void
WithdrawalRequest
- requestId: String
- application: InternshipApplication
- requestedBy: Student
- requestedOn: LocalDate
- reason: String
- status: WithdrawalRequestStatus = PENDING
- processedBy: CareerCenterStaff
- processedOn: LocalDate
- staffNote: String
+ getRequestId(): String
+ getApplication(): InternshipApplication
+ getRequestedBy(): Student
+ getRequestedOn(): LocalDate
+ getReason(): String
+ getStatus(): WithdrawalRequestStatus
+ getProcessedBy(): CareerCenterStaff
+ getProcessedOn(): LocalDate
+ getStaffNote(): String
+ isPending(): boolean
+ setReason(reason: String): void
+ approve(staff: CareerCenterStaff, note: String): void
+ reject(staff: CareerCenterStaff, note: String): void
+ ensurePending(): void
+ stamp(staff: CareerCenterStaff, note: String): void
InternshipApplication
- applicationId: String
- appliedOn: LocalDate
- student: Student
- internshipId: String
- internship: Internship
- studentAccepted: boolean
- status: ApplicationStatus = PENDING
+ getApplicationId(): String
+ getAppliedOn(): LocalDate
+ getStudent(): Student
+ getInternshipId(): String
+ getInternship(): Internship
+ getStatus(): ApplicationStatus
+ isStudentAccepted(): boolean
+ canAccept(): boolean
+ markAccepted(): boolean
+ markSuccessful(): void
+ markUnsuccessful(): void
+ markWithdrawn(): void
+ confirmAcceptance(appReadPort: Student.AppReadPort): void
+ reviseAcceptance(forApprovedWithdrawal): void
+ setApplicationStatus(applicationId: String): void
+ setStudentAccepted(studentAccepted): void
+ setInternship(internship: Internship): void
+ ensureStatus(expected: ApplicationStatus, msg: String): void
Internship
- internshipId: String
- title: String
- description: String
- level: InternshipLevel
- preferredMajor: String
- openDate: LocalDate
- closeDate: LocalDate
- companyName: String
- maxSlots: int
- confirmedSlots: int
- visible: boolean
- status: InternshipStatus = PENDING
- major: String
- yearOfStudy: int
- email: String
+ getInternshipId(): String
+ getTitle(): String
+ getDescription(): String
+ getLevel(): InternshipLevel
+ getPreferredMajor(): String
+ getOpenDate(): LocalDate
+ getCloseDate(): LocalDate
+ getCompanyName(): String
+ getMaxSlots(): int
+ getConfirmedSlots(): int
+ isVisible(): boolean
+ getStatus(): InternshipStatus
+ publish(): void
+ reject(): void
+ setVisible(visible: boolean): void
+ isOpenForApplications(now: LocalDate): boolean
+ incrementConfirmedSlots(): void
+ decrementConfirmedSlots(): void
+ isEditable(): boolean
<<interface>>
CrudRepository<T, ID>
+ findById(id: ID): Optional<T>
+ findAll(): List<T>
+ save(entity: T): T
+ saveAll(entities: Iterable<T>): List<T>
+ deleteById(id: ID): void
+ deleteAllById(ids: Iterable<ID>): void
+ deleteAll(): void
+ existsById(id: ID): boolean
+ count(): long
<<interface>>
Student.AppReadPort
+ countActiveApplications(studentId: String) : int
+ hasConfirmedPlacement(studentId: String) : boolean
<<uses>>
<<uses>>
<<uses>>
<<uses>>
CompanyRepRepository
- map : Map<String, CompanyRep>
+ findById(email: String) : Optional<CompanyRep>
+ findAll() : List<CompanyRep>
+ save(rep: CompanyRep) : CompanyRep
+ saveAll(entities: Iterable<CompanyRep>) : List<CompanyRep>
+ deleteById(email: String) : void
+ deleteAllById(emails: Iterable<String>) : void
+ deleteAll() : void
+ existsById(email: String) : boolean
+ count() : long
+ findByEmail(email: String) : Optional<CompanyRep>
+ saveToCsv(path: String) : void
+ loadFromCsv(path: String) : void
- getCsvValue(row: String[], idx: Map<String, Integer>, possibleHeaders: String...) : String
CareerCenterStaffRepository
- map : Map<String, CareerCenterStaff>
+ findById(id: String) : Optional<CareerCenterStaff>
+ findAll() : List<CareerCenterStaff>
+ save(s: CareerCenterStaff) : CareerCenterStaff
+ saveAll(entities: Iterable<CareerCenterStaff>) : List<CareerCenterStaff>
+ deleteById(id: String) : void
+ deleteAllById(ids: Iterable<String>) : void
+ deleteAll() : void
+ existsById(id: String) : boolean
+ count() : long
+ loadFromCsv(path: String) : void
+ saveToCsv(path: String) : void
+ findByDepartment(department: String) : List<CareerCenterStaff>
- getCsvValue(row: String[], idx: Map<String, Integer>, possibleHeaders: String...) : String
StudentRepository
- map : Map<String, Student>
+ findById(id: String) : Optional<Student>
+ findAll() : List<Student>
+ save(entity: Student) : Student
+ saveAll(entities: Iterable<Student>) : List<Student>
+ deleteById(id: String) : void
+ deleteAllById(ids: Iterable<String>) : void
+ deleteAll() : void
+ existsById(id: String) : boolean
+ count() : long
+ findByMajor(major: String) : List<Student>
+ findByYear(year: int) : List<Student>
+ loadFromCsv(path: String) : void
+ saveToCsv(path: String) : void
- getCsvValue(row: String[], idx: Map<String, Integer>, possibleHeaders: String...) : String
WithdrawalRequestRepository
- map : Map<String, WithdrawalRequest>
+ findById(id: String) : Optional<WithdrawalRequest>
+ findAll() : List<WithdrawalRequest>
+ save(wr: WithdrawalRequest) : WithdrawalRequest
+ saveAll(entities: Iterable<WithdrawalRequest>) : List<WithdrawalRequest>
+ deleteById(id: String) : void
+ deleteAllById(ids: Iterable<String>) : void
+ deleteAll() : void
+ existsById(id: String) : boolean
+ count() : long
+ findByStudent(studentId: String) : List<WithdrawalRequest>
+ findByApplicationId(applicationId: String) : Optional<WithdrawalRequest>
+ findPending() : List<WithdrawalRequest>
+ countPendingForInternship(internshipId: String) : long
+ saveToCsv(path: String) : void
+ loadFromCsv(path: String, applications: ApplicationRepository, students: StudentRepository, staffRepo: CareerCenterStaffRepository) : void
- getCsvValue(row: String[], idx: Map<String, Integer>, possibleHeaders: String...) : String
ApplicationRepository
- map : Map<String, InternshipApplication>
+ findById(id: String) : Optional<InternshipApplication>
+ findAll() : List<InternshipApplication>
+ save(app: InternshipApplication) : InternshipApplication
+ saveAll(entities: Iterable<InternshipApplication>) : List<InternshipApplication>
+ deleteById(id: String) : void
+ deleteAllById(ids: Iterable<String>) : void
+ deleteAll() : void
+ existsById(id: String) : boolean
+ count() : long
+ existsByStudentAndInternship(studentId: String, internshipId: String) : boolean
+ countActiveByStudent(studentId: String) : long
+ countAcceptedForInternship(internshipId: String) : long
+ findByStudent(studentId: String) : List<InternshipApplication>
+ findByInternship(internshipId: String) : List<InternshipApplication>
+ deleteByStudent(studentId: String) : void
+ clear() : void
+ countActiveApplications(studentId: String) : int
+ hasConfirmedPlacement(studentId: String) : boolean
+ saveToCsv(path: String) : void throws IOException
+ loadFromCsv(path: String, students: StudentRepository, internships: InternshipRepository) : void throws IOException
- getCsvValue(row: String[], idx: Map<String, Integer>, possibleHeaders: String...) : String
InternshipRepository
- map : Map<String, Internship>
+ findById(id: String) : Optional<Internship>
+ findAll() : List<Internship>
+ save(entity: Internship) : Internship
+ saveAll(entities: Iterable<Internship>) : List<Internship>
+ deleteById(id: String) : void
+ deleteAllById(ids: Iterable<String>) : void
+ deleteAll() : void
+ existsById(id: String) : boolean
+ count() : long
+ findByStatus(status: InternshipStatus) : List<Internship>
+ findByLevel(level: InternshipLevel) : List<Internship>
+ findByPreferredMajor(major: String) : List<Internship>
+ findOpenVisibleForDate(date: LocalDate) : List<Internship>
+ findAllVisible() : List<Internship>
+ countApproved() : long
+ findByCompanyName(companyName: String) : List<Internship>
+ findActiveByCompanyName(companyName: String) : List<Internship>
+ saveToCsv(path: String) : void
+ loadFromCsv(path: String) : void
- getCsvValue(row: String[], idx: Map<String, Integer>, possibleHeaders: String...) : String
BaseController
studentRepo: StudentRepository
repRepo: CompanyRepRepository
staffRepo: CareerCenterStaffRepository
internshipRepo: InternshipRapository
applicationRepo: ApplicationRepository
withdrawalRepo: WithdrawalRequestRepository
isEmailValid(email: String): boolean
isNullOrEmpty(str: String): boolean
validateNotNegative(value: int, fieldName: String): void
<<enumeration>>
Role
STUDENT
COMPANY_REP
STAFF
<<uses>>
<<uses>>
<<uses>>
<<uses>>
<<uses>>
AuthController
- pw : Map<String, String>
+ seedDefaults(students: StudentRepository, reps: CompanyRepRepository, staff: CareerCenterStaffRepository) : void
+ login(loginKey: String, password: String) : String
+ changePassword(loginKey: String, oldPw: String, newPw: String) : void
+ verify(loginKey: String, password: String): boolean
+ getUserRole(loginKey: String): Role
- verifyAndSet(authKey: String, oldPw: String, newPw: String) : void
- requireReps() : void
- key(r: Role, id: String): String
- normEmail(email: String): String
CompanyRepRegistrationController
+ registerCompanyRep(name: String, companyName: String, department: String, position: String, email: String) : String
+ emailExists(email: String): boolean
+ getAllCompanyReps(): List<CompanyRep>
- getRepByEmail(email: String) : CompanyRep
- validateRegistrationFields(name: String, companyName: String, department: String, position: String, email: String) : void
- isValidEmail(email: String) : boolean
CompanyRepcontroller
+ listInternships(repEmail: String) : List<Internship>
+ getInternship(repEmail: String, internshipId: String) : Internship
+ createInternship(repEmail: String, title: String, description: String, level: InternshipLevel, preferredMajor: String, openDate: LocalDate, closeDate: LocalDate, maxSlots: int) : String
+ editInternship(repEmail: String, internshipId: String, title: String, description: String, level: InternshipLevel, preferredMajor: String, openDate: LocalDate, closeDate: LocalDate, maxSlots: int) : void
+ deleteInternship(repEmail: String, internshipId: String) : void
+ setVisibility(repEmail: String, internshipId: String, visible: boolean) : void
+ listApplications(repEmail: String, internshipId: String) : List<InternshipApplication>
+ acceptApplication(repEmail: String, applicationId: String) : void
+ markApplicationUnsuccessful(repEmail: String, applicationId: String) : void
- validateRepExists(repEmail: String) : CompanyRep
- validateOwnership(rep: CompanyRep, internship: Internship) : void
- validateInternshipParams(title: String, description: String, level: InternshipLevel, preferredMajor: String, openDate: LocalDate, closeDate: LocalDate, maxSlots: int) : void
CareerCenterStaffController
+ listPendingInternships() : List<Internship>
+ listUnapprovedReps() : List<CompanyRep>
+ listAllReps() : List<CompanyRep>
+ listPendingWithdrawals() : List<WithdrawalRequest>
+ listAllInternships() : List<Internship>
+ approveInternship(staffId: String, internshipId: String, makeVisible: boolean) : void
+ rejectInternship(staffId: String, internshipId: String) : void
+ approveCompanyRep(staffId: String, repEmail: String) : void
+ rejectCompanyRep(staffId: String, repEmail: String, reason: String) : void
+ approveWithdrawal(staffId: String, wrId: String, note: String) : void
+ rejectWithdrawal(staffId: String, wrId: String, note: String) : void
+ filterInternships(status: InternshipStatus, major: String, companyName: String, level: String) : List<Internship>
- validateStaffExists(staffId: String) : CareerCenterStaff
StudentController
+ getStudent(studentId: String): Student
+ viewEligibleInternships(studentId: String): List<Internship>
+ viewFilteredInternships(studentId: String, major: String, level: InternshipLevel, companyName: String): List<Internship>
+ applyForInternship(studentId: String, internshipId: String): String
+ withdrawApplication(studentId: String, applicationId: String): void
+ requestWithdrawal(studentId: String, applicationId: String, reason: String): String
+ viewMyApplications(studentId: String) : List<InternshipApplication>
+ viewMyWithdrawalRequests(studentId: String): List<WithdrawalRequest>
+ getApplication(studentId: String, applicationId: String): InternshipApplication
+ canApplyMore(studentId: String): boolean
+ getActiveApplicationCount(studentId: String): int
+ hasConfirmedPlacement(studentId: String): boolean
- getInternship(internshipId: String): Internship
DataSaveController
- baseDir : Path
+ saveAll(studentsCsv: String, repsCsv: String, staffCsv: String, internshipsCsv: String, applicationsCsv: String, withdrawalsCsv: String): void
+ loadAll(studentsCsv: String, repsCsv: String, staffCsv: String, internshipsCsv: String, applicationsCsv: String, withdrawalsCsv: String): void
+ getBaseDir(): Path
+ isInitialized(): boolean
LoginBoundary.Session
- students: StudentRepository
- reps: CompanyRepRepository
- staff: CareerCenterStaffRepository
- auth: AuthController
+ performLogin() : Optional<Session>
+ login(userInput: String) : Optional<Session>
+ displayLoginHelp() : void
- getPassword() : String
- createSessionFromAuthToken(authToken: String, userInput: String) : Optional<Session>
- promptRetry() : boolean
LoginBoundary
- students: StudentRepository
- reps: CompanyRepRepository
- staff: CareerCenterStaffRepository
- auth: AuthController
+ performLogin() : Optional<Session>
+ login(userInput: String) : Optional<Session>
+ displayLoginHelp() : void
- getPassword() : String
- createSessionFromAuthToken(authToken: String, userInput: String) : Optional<Session>
- promptRetry() : boolean
<<enumeration>>
Role
STUDENT
COMPANY_REP
STAFF
CompanyRepRegistrationBoundary
- ctl : CompanyRepRegistrationController
+ signupCompanyRep() : void
- promptNonEmpty(prompt: String): String
- promptEmail(): String
CompanyRepBoundary
-ctl: CompanyRepController
+ menu(repEmail: String): void
- handleInternships(repEmail: String): void
- handleInternshipActions(repEmail: String, internship: Internship): void
- handleApplications(repEmail: String, internship: Internship): void
- displayApplications(appsIn: List<InternshipApplication>): void
- handleApplicationActions(repEmail: String, application: InternshipApplication): void
- handleCreateInternship(repEmail: String): void
- handleEditInternship(repEmail: String, internship: Internship): void
- editInternshipDetails(repEmail: String, internship: Internship): void
- handleDeleteInternship(repEmail: String, internship: Internship): void
- getInternshipLevel(): InternshipLevel
- getOptionalInternshipLevel(currentLevel: InternshipLevel): InternshipLevel
- getDateInput(prompt: String, allowPast: boolean): LocalDate
- getOptionalDateInput(prompt: String, currentDate : LocalDate, allowPast: boolean): LocalDate
- getStringInput(prompt: String): String
- getOptionalStrInput(prompt: String, currentStrIn: int): int
- displayInternshipDetails(internship: Internship): void
CareerCenterStaffBoundary
-ctl: CareerCenterStaffController
+ menu(staffId: String) : void
- handlePendingInternships(staffId: String): void
- handlePendingCompanyReps(staffId: String) : void
- handlePendingWithdrawals(staffId: String) : void
- handleReports(staffId: String) : void
- generateInternshipReport() : void
- generateInternshipsByMajor() : void
- generateInternshipsByCompany() : void
- generateInternshipsByLevel() : void
- generateAllInternships() : void
- displayInternshipReport(internships: List<Internship>, filter: String) : void
- displayInternshipDetails(internship: Internship) : void
- displayRepDetails(rep: CompanyRep) : void
StudentBoundary
- ctl: StudentController
+ menu(studentId: String) : void
- handleAvailableInternships(studentId: String) : void
- handleMyApplications(studentId: String) : void
- handleApplicationStatusView(studentId: String, allApps: List<InternshipApplication>, status: ApplicationStatus, title: String) : void
- handleAllApplicationsView(studentId: String, allApps: List<InternshipApplication>) : void
- handleApplicationManagement(studentId: String, app: InternshipApplication) : void
- handleWithdrawalRequest(studentId: String, app: InternshipApplication) : void
- handleMyWithdrawalRequests(studentId: String) : void
- displayWithdrawalDetails(wr: WithdrawalRequest) : void
- displayInternshipDetails(internship: Internship) : void
BaseBoundary
- sc: Scanner
- auth: AuthController
getRequiredInput(prompt: String): String
getOptionalInput(prompt: String, currentValue: String): String
safeInt(s: String, def: int): int
isNullOrEmpty(str: String): boolean
isCancelCommand(input: String): boolean
displayCancelMessage() : void
changePassword(loginKey: String): boolean
displaySectionHeader(title: String): void
MainMenuBoundary
- login: LoginBoundary
- studentUI: StudentBoundary
- repUI: CompanyRepBoundary
- staffUI: CareerCenterStaffBoundary
- registrationUI: CompanyRepRegistrationBoundary
+ run() : void
- handleLogin() : void
- confirmExit() : boolean